Schedule Intelligence

AI-Powered Meeting & Workload Agent

INTERNAL TECHNICAL DOCUMENTATION

Slipstream Data · Built on Snowflake Cortex

Version 1.0 · 2026

Zuhayr Adams

1. What Is Schedule Intelligence?

Schedule Intelligence is an AI-powered chatbot built on Snowflake's Cortex platform that allows anyone at Slipstream Data to ask natural language questions about employee meeting schedules and get back instant answers, complete with data tables, charts, and plain-English analysis.

Instead of writing SQL queries or building pivot tables, you simply ask questions like 'Show me Palesa's meetings this month' or 'Who has the busiest Mondays?' and the agent figures out the rest.

> **What makes this different from a regular dashboard?**
> A dashboard shows you what it was built to show. This agent answers questions that were never anticipated at build time. You can ask anything in natural language and it will attempt to answer it from your data in real time.

1.1 Key Capabilities

- Natural language question answering about any employee's schedule
- Automatic SQL generation: you never write a single query
- Interactive charts auto-rendered based on what the data contains
- Plain-English AI analysis summarising the key findings
- Pronoun and context resolution: ask 'is she busy that day?' after asking about a specific person
- Conversation memory: the agent tracks what you asked previously in the same session
- Quick prompt shortcuts for common questions
- Session stats tracking queries, rows fetched, and charts generated

2. How The Agent Was Built

The agent was built iteratively, starting with a basic Streamlit app and progressively adding intelligence, better error handling, and richer UI. Below is the journey from idea to working product.

2.1 Starting Point: The Idea

The goal was to make employee schedule data conversational. Rather than requiring someone to understand SQL or navigate a BI tool, the idea was to let anyone ask questions in plain English directly inside Snowflake's environment.

Snowflake's Cortex Analyst service was identified as the right tool, it takes a semantic model (a YAML description of your data) and translates natural language questions into SQL automatically, using large language models under the hood.

2.2 The Technology Stack

Component	Role
Streamlit in Snowflake	The front-end web application framework, hosted natively inside Snowflake
Snowflake Cortex Analyst	The NL-to-SQL engine that powers question answering
Snowflake Cortex Complete (mistral-large2)	The LLM used for pronoun resolution and natural language summaries
Snowpark Python Session	Used to execute generated SQL and return results as DataFrames
Plotly Express	Used to auto-render charts based on the columns returned
_snowflake module	Internal Snowflake module for calling the Cortex Analyst REST API
YAML Semantic Model	A structured description of the data that Cortex Analyst uses to generate SQL

2.3 Build Journey - Step by Step

The following steps reflect the actual order in which the agent was built and debugged:

Step 1: Basic Cortex Complete App

The first version used SNOWFLAKE.CORTEX.COMPLETE with the snowflake-arctic model. It was a simple chat interface but had a critical flaw: it was just an LLM with no awareness of the actual data. It could not query the database at all.

Step 2: Switching to Cortex Analyst

The core switch was from CORTEX.COMPLETE to the Cortex Analyst REST API endpoint (/api/v2/cortex/analyst/message) using the _snowflake.send_snow_api_request() function. This is what gave the agent the ability to actually generate SQL from natural language and run it against the database.

> **Why not use session.call()?**
> The first attempt used session.call('SNOWFLAKE.CORTEX.ANALYST', ...) which failed with an 'Unknown user-defined function' error. Cortex Analyst is not a stored procedure — it is a REST API service and must be called via the internal HTTP request mechanism.

Step 3: Fixing the YAML Semantic Model

Several rounds of YAML debugging were needed before Cortex Analyst could understand the data:

- The agg field for measures was invalid, the correct field is default_aggregation
- average was not a valid enum value, the correct value is avg
- The base_table field was formatted as an array instead of an object with database, schema, and table keys
- The SCHEDULE_DATE column was originally named DATE in the YAML, not matching the actual column name
- The meeting_count measure used MEETING_SUBJECT (a text column) as its expression, causing a numeric conversion error and it was fixed by using expr: '1' so each row counts as one meeting

Step 4: Adding Natural Language Summaries

Cortex Analyst returns SQL and a brief interpretation, but no narrative analysis. A second LLM call was added using SNOWFLAKE.CORTEX.COMPLETE with mistral-large2 to generate a 3-5 sentence plain-English summary of the query results after every response.

Step 5: Solving the Pronoun Problem

When a user asked 'Show me Rosy's meetings' and then followed up with 'Is she busy that day?', the agent had no idea who 'she' referred to or what 'that day' meant. Cortex Analyst is a SQL generator, not a conversational LLM as it cannot resolve pronouns on its own.

The solution was a pre-processing step: before every Cortex Analyst call, a Cortex Complete call rewrites the user's question into a fully explicit, self-contained question using the conversation history. So 'Is she busy that day?' becomes 'Is Rosy busy on 2025-11-11?' before it ever reaches the SQL engine.

> **Why not use Cortex Analyst's native multi-turn support?**
> Cortex Analyst only accepts the role 'user' in its messages array — it does not support 'assistant' roles. Any attempt to include assistant messages in the history results in a 400 error. The pre-rewriting approach using Cortex Complete was the correct workaround.

Step 6: UI Polish

The final phase focused on making the application visually polished and user-friendly using custom CSS with a dark theme, DM Serif Display and DM Sans fonts, auto-rendered Plotly charts, metric cards, a persistent sidebar, quick prompt buttons, and session stats.

3. The Semantic Model

The semantic model is the most important piece of the entire agent. It is a YAML file that describes your database table in plain English,the column names, what they mean, sample values, and how they should be aggregated. Without a good semantic model, Cortex Analyst cannot generate correct SQL.

3.1 What Is a Semantic Model?

Think of the semantic model as a translator between human language and your database schema. When you ask, 'How many meetings did Andrew have this week?', Cortex Analyst reads the semantic model to understand:

- Which table to query (EMPLOYEE_SCHEDULE)
- Which column represents employee name (FIRST_NAME, labelled as employee_name)
- Which column represents the date (SCHEDULE_DATE, typed as a time dimension)
- What 'count of meetings' means (the meeting_count measure, which sums the expression '1')

3.2 Structure of the YAML File

The semantic model file is stored in a Snowflake stage and referenced by the Cortex Analyst API. Here is the structure used for the Schedule Intelligence agent:

Top-Level Fields

Field	Purpose
name	A unique identifier for the semantic model
description	A plain English description of what the model covers
tables	A list of database tables included in the model

Table-Level Fields

Field	Purpose
name	Friendly name for the table used internally by the model

Field	Purpose
base_table	Object with database, schema, and table keys pointing to the actual Snowflake table
description	Explains what each row represents and any important nuances
dimensions	Text/categorical columns (names, categories, labels)
time_dimensions	Date and timestamp columns
measures	Numeric columns with aggregation rules

Dimension Fields

Field	Purpose
name	Friendly name shown to the LLM
expr	The actual SQL column name in the database
data_type	TEXT, NUMBER, DATE, or TIMESTAMP
description	Plain English explanation of what the column contains
sample_values	Example values to help the LLM understand the data

Measure Fields

Field	Purpose
name	Friendly name for the measure
expr	The SQL expression (column name or literal like '1')
data_type	Always NUMBER for measures
default_aggregation	How to aggregate: sum, avg, or count
description	Explains what this measure represents

3.3 The Verified Queries Section

One of the most powerful features of the semantic model is the verified_queries section. These are example question-and-SQL pairs that you write yourself and include in the YAML. They act as ground truth examples that teach Cortex Analyst exactly how to write SQL for your specific data patterns.

For example, the verified query for 'Compare the workload between Robin and Andrew' shows Cortex Analyst that it should GROUP BY first_name, SUM duration_minutes, and AVG load_percentage by teaching it the pattern for all future comparison questions.

> **Tip: Invest in verified_queries**
> The more verified_queries you provide, the more accurate Cortex Analyst becomes. Start with your most common question types and add more as you discover edge cases. Each verified query dramatically improves the accuracy of similar questions.

3.4 Common YAML Mistakes to Avoid

- Using agg instead of default_aggregation - the correct field name is default_aggregation
- Using average instead of avg as the aggregation type - valid values are sum, avg, and count
- Formatting base_table as an array [DB, SCHEMA, TABLE] instead of an object with database, schema, and table keys
- Mismatching column names - the expr field must exactly match the actual SQL column name including case
- Using a text column as the expr for a measure - use a numeric column or the literal '1' for row counts
- Compressing the YAML file when uploading to the stage - always use AUTO_COMPRESS = FALSE

4. Creating the Agent in Snowflake

There are two ways to set up a Cortex Analyst agent in Snowflake. The first is through the Snowflake UI using the native agent creation wizard. The second is by writing your own Streamlit application and calling the API directly, which is what Schedule Intelligence does.

4.1 Method 1: Using the Snowflake UI Agent Builder

Snowflake provides a built-in way to create and test a Cortex Analyst agent through the Snowsight interface. Follow these steps:

Step 1: Upload Your Semantic Model

Before creating the agent, your YAML file must be in a Snowflake stage. Run the following in a worksheet:

```
CREATE STAGE IF NOT EXISTS MY_AI_MODELS;
```

```
PUT file:///path/to/Employee_Schedule.yaml @MY_AI_MODELS OVERWRITE=TRUE
AUTO_COMPRESS=FALSE;
```

Verify it uploaded correctly:

```
LIST @SCHEDULE_DB.PUBLIC.MY_AI_MODELS;
SELECT $1 FROM @SCHEDULE_DB.PUBLIC.MY_AI_MODELS/Employee_Schedule.yaml;
```

Step 2: Navigate to Cortex Analyst in Snowsight

- Log into Snowsight (the Snowflake web UI)
- In the left navigation, go to AI & ML > Cortex Analyst
- Click Create Agent or New Conversation

Step 3: Select Your Semantic Model

- When prompted, select the stage where your YAML is stored
- Select the specific YAML file (Employee_Schedule.yaml)
- Snowflake will validate the YAML and fix any errors shown before proceeding

Step 4: Test the Agent

- A chat interface will appear where you can test questions directly
- Try questions like 'Show me all meetings for Andrew' to verify the SQL is correct
- Review the generated SQL to make sure it matches your expectations
- If results are wrong, go back and improve your semantic model YAML

4.2 Method 2: Calling the API Directly in Streamlit

This is the approach used by Schedule Intelligence. Instead of using the built-in UI, you call the Cortex Analyst REST API directly from a Streamlit app using the _snowflake module's send_snow_api_request function.

> **Why this approach?**
> Calling the API directly gives you full control over the UI, the ability to add custom logic like pronoun resolution and LLM summaries, and the ability to build a branded, polished experience that looks nothing like a standard Snowflake interface.

The API Call

The core API call looks like this:

```
resp = _snowflake.send_snow_api_request(
    'POST',
    '/api/v2/cortex/analyst/message',
    {},   # headers
    {},   # query params
    {
        'messages': [{'role': 'user', 'content': [{'type': 'text', 'text':
question}]}],
        'semantic_model_file':
'@SCHEDULE_DB.PUBLIC.MY_AI_MODELS/Employee_Schedule.yaml'
    },
    None,
    10000  # timeout in milliseconds
)
```

The response contains content blocks. A text block contains the interpretation, and an sql block contains the generated SQL statement that you then run against the database using session.sql().

5. The Streamlit Application

The Streamlit app is the user-facing interface. It is a single Python file (streamlit_app.py) deployed inside Snowflake's Streamlit in Snowflake environment. Below is a walkthrough of how each part works.

5.1 Application Architecture

The app follows a three-stage pipeline for every question:

Stage	What Happens
Stage 1 — Pronoun Resolution	Cortex Complete (mistral-large2) rewrites the user's question to be fully explicit, resolving any pronouns or vague references using the conversation history
Stage 2 — SQL Generation	The rewritten question is sent to Cortex Analyst which returns a SQL statement and a brief interpretation
Stage 3 — Analysis & Display	The SQL is executed, results are displayed as a table and charts, then Cortex Complete generates a natural language summary of the findings

5.2 Pronoun Resolution

This is one of the most important features of the app. Because Cortex Analyst is purely a SQL generator, it cannot resolve conversational references like pronouns or relative time references.

The resolve_pronouns() function calls mistral-large2 with the last 8 messages of conversation history and the current question, asking it to rewrite the question as a fully self-contained query. The rewritten question is then shown to the user in a green banner so they can see exactly what the agent understood.

Example: User asks 'Show me Palesa's meetings', then follows up with 'Is she busy on that day?' then the agent rewrites this to 'Is Palesa busy on 2025-11-15?' before querying the database.

5.3 Auto-Generated Charts

The render_charts() function inspects the columns returned by the SQL query and automatically decides which charts to render based on what data is present. No configuration is needed — the charts appear automatically.

Columns Detected	Chart Rendered
EMPLOYEE_NAME + TOTAL_MEETING_MINUTES	Bar chart comparing total meeting minutes per employee
SCHEDULE_DATE + MEETING_TITLE	Bar chart of meetings per day
MEETING_TITLE + TOTAL_MINUTES	Horizontal bar chart of top meetings by duration
TIME_OF_DAY	Donut chart of meetings by time of day
WORKLOAD_BAND	Donut chart of workload distribution
SCHEDULE_DATE + LOAD_PERCENTAGE	Line chart of load percentage over time
DAY_OF_WEEK	Bar chart of meetings by day of week
MONTH_NAME + TOTAL_MINUTES	Line chart of meeting minutes by month

5.4 Natural Language Summary

After the data is displayed, the get_nl_summary() function sends the original question and the first 50 rows of results to mistral-large2 and asks it to write a 3-5 sentence plain-English analysis of the findings, highlighting patterns, outliers, and specific numbers.

This summary is displayed in a styled card at the bottom of each response, clearly labelled as AI Analysis so users understand it is an interpretation rather than raw data.

5.5 Session State and Stats

Streamlit rerenders the entire page on every interaction. The app uses st.session_state to persist the conversation history, query count, total rows fetched, and charts generated across rerenders. The sidebar displays these stats live so users can track their session activity.

5.6 The Sidebar

The sidebar is always visible and contains four sections: a live connection status indicator, the semantic model reference, session statistics, and eight quick prompt buttons for common questions. A Clear Chat button resets the session entirely. The sidebar is forced open via CSS so it cannot be accidentally collapsed.

6. The Language Models Used

Two different language models are used in the agent, each chosen for a different purpose:

Model	Role & Reasoning
Snowflake Cortex Analyst	The core NL-to-SQL engine. Not a general LLM — it is a specialised system trained specifically to translate natural language questions into SQL using a provided semantic model. It understands table structures, column types, and aggregation rules from the YAML.
mistral-large2	A general-purpose large language model accessed via SNOWFLAKE.CORTEX.COMPLETE. Used for two tasks: (1) rewriting ambiguous questions into explicit ones before they reach Cortex Analyst, and (2) generating natural language analysis summaries of the query results.

> **Why mistral-large2 for summaries?**
> mistral-large2 was chosen over other available models because it produces fluent, concise, and factually grounded summaries when given structured data as CSV input. It follows instructions reliably — in particular, the instruction to not use bullet points and to write in conversational prose.

7. How to Update and Maintain the Agent

7.1 Adding New Columns to the Semantic Model

If new columns are added to the EMPLOYEE_SCHEDULE table, the semantic model YAML must be updated to include them. Open the YAML, add the new field under dimensions, time_dimensions, or measures as appropriate, and re-upload to the stage with OVERWRITE=TRUE AUTO_COMPRESS=FALSE.

7.2 Improving Query Accuracy

If the agent gives wrong answers to certain questions, the best fix is to add a verified_query entry to the YAML for that question type. Write the correct SQL yourself, add it to the

verified_queries section, and re-upload. Cortex Analyst will use it as a reference for similar future questions.

7.3 Adding New Quick Prompts

Quick prompts are defined in the suggestions list inside the sidebar section of streamlit_app.py. Add any string to this list and it will appear as a clickable button.

7.4 Switching LLM Models

The model name mistral-large2 appears in two places in the app — the resolve_pronouns() function and the get_nl_summary() function. To switch to a different Cortex-supported model, change the model name string in both function calls.

7.5 Uploading a New Version of the Streamlit App

- Navigate to Snowsight > Streamlit
- Find the Schedule Intelligence app
- Click Edit and paste the updated Python code, or upload the new file
- Click Run to deploy the changes

8. Quick Reference

8.1 Key File Locations

Asset	Location
Semantic Model YAML	@SCHEDULE_DB.PUBLIC.MY_AI_MODELS/Employee_Schedule.yaml
Streamlit App	Snowsight > Streamlit > Schedule Intelligence
Source Table	SCHEDULE_DB.PUBLIC.EMPLOYEE_SCHEDULE
API Endpoint	/api/v2/cortex/analyst/message

8.2 Example Questions the Agent Can Answer

- Show me all of Palesa's meetings in November
- Compare the workload between Robin and Andrew
- Who has the most meetings on Mondays?
- What percentage of Andrew's day is spent in meetings on average?
- Show me all High workload days across the team
- Which month had the most total meeting hours?
- What time of day do most meetings happen?
- Give me a monthly summary of meeting hours per employee

8.3 Troubleshooting

Issue	Solution
Agent says it cannot find the employee	Check that the name is spelled exactly as it appears in the FIRST_NAME column. Names are case-sensitive in SQL.
YAML validation error on upload	Check that the file was uploaded with AUTO_COMPRESS=FALSE and that all field names match the spec exactly (e.g. default_aggregation not agg)
SQL execution error about numeric conversion	A measure's expr is pointing to a text column. Use a numeric column or the literal '1' for counts.
Pronoun resolution not working	The conversation history may be too old or unclear. Rephrase the question to be explicit, or click Clear Chat and start a new session.
API Error 400 — role must change	The messages array contains consecutive user messages. This is handled automatically in the current code but may resurface if the session state gets corrupted. Click Clear Chat.
Charts not appearing	Charts only render when specific column combinations are present. Compare questions return different columns than detail questions, so different charts appear.

— End of Document —